

Big Data and Automated Content Analysis (6EC)

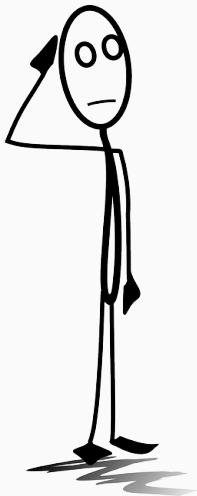
Week 5: »Unsupervised machine learning« Thursday

Anne Kroon
a.c.kroon@uva.nl

November 27, 2025

UvA RM Communication Science

Appendix: Code examples



*Everything clear from previous weeks?
Questions?*

This week, we will get a general overview of working with textual data. Due to a lack of time, I will introduce you to some of the basic concepts, point you to resources, and give you a practical, hands-on introduction.

Boumans and Trilling, 2016: Types of Automated Content Analysis

	Methodological approach		
	<i>Counting and Dictionary</i>	<i>Supervised Machine Learning</i>	<i>Unsupervised Machine Learning</i>
Typical research interests and content features	visibility analysis sentiment analysis subjectivity analysis	frames topics gender bias	frames topics
Common statistical procedures	string comparisons counting	support vector machines naive Bayes	principal component analysis cluster analysis latent dirichlet allocation semantic network analysis

deductive

inductive

Bottom-up vs. top-down

Bottom-up

- Count most frequently occurring words
- Maybe better: Count combinations of words $\Rightarrow$ Which words co-occur together?

We *don't* specify what to look for in advance

Bottom-up vs. top-down

Bottom-up

- Count most frequently occurring words
- Maybe better: Count combinations of words $\Rightarrow$ Which words co-occur together?

We *don't* specify what to look for in advance

Top-down

- Count frequencies of pre-defined words
- Maybe better: patterns instead of words

We *do* specify what to look for in advance

A simple bottom-up approach

```
1 from collections import Counter
2 texts = ["Communication in the Digital Society is a very very complex
↪ phenomenon", "I like to study it"]
3 bottom_up = []
4 for t in texts:
5     bottom_up.append(Counter(t.lower().split()).most_common(3))
6     print(bottom_up)
```


A simple bottom-up approach

```
1 from collections import Counter
2 texts = ["Communication in the Digital Society is a very very complex
  ↪ phenomenon", "I like to study it"]
3 bottom_up = []
4 for t in texts:
5     bottom_up.append(Counter(t.lower().split()).most_common(3))
6     print(bottom_up)
```

This results in:

```
1 [('very', 2), ('Communication', 1), ('in', 1)]
2 [('I', 1), ('like', 1), ('to', 1)]
```

Please note that you can also write this like:

```
1 bottom_up = [Counter(t.split()).most_common(3) for t in texts]
```

A simple bottom-up approach

```
1 from collections import Counter
2 texts = ["Communication in the Digital Society is a very very complex
↪ phenomenon", "I like to study it"]
3 bottom_up = []
4 for t in texts:
5     bottom_up.append(Counter(t.lower().split()).most_common(3))
6     print(bottom_up)
```

This results in:

```
1 [('very', 2), ('Communication', 1), ('in', 1)]
2 [('I', 1), ('like', 1), ('to', 1)]
```

Please note that you can also write this like:

```
1 bottom_up = [Counter(t.split()).most_common(3) for t in texts]
```

A simple top-down approach

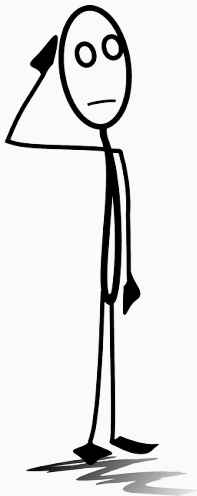
```
1 texts = ["Communication in the Digital Society is a very very complex  
↪ phenomenon", "I like to study it"]  
2 features = ["communication", "digital", "study"]  
3 for t in texts:  
4     print(f"\nAnalyzing '{t}':")  
5     for f in features:  
6         print(f"{f} occurs {t.lower().count(f)} times")
```

```
1 Analyzing 'Communication in the Digital Society is a very very complex phenomenon':  
2 communication occurs 1 times  
3 digital occurs 1 times  
4 study occurs 0 times  
5  
6 Analyzing 'I like to study it':  
7 communication occurs 0 times  
8 digital occurs 0 times  
9 study occurs 1 times
```

A simple top-down approach

```
1 texts = ["Communication in the Digital Society is a very very complex  
↪ phenomenon", "I like to study it"]  
2 features = ["communication", "digital", "study"]  
3 for t in texts:  
4     print(f"\nAnalyzing '{t}':")  
5     for f in features:  
6         print(f"{f} occurs {t.lower().count(f)} times")
```

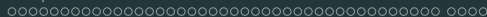
```
1 Analyzing 'Communication in the Digital Society is a very very complex phenomenon':  
2 communication occurs 1 times  
3 digital occurs 1 times  
4 study occurs 0 times  
5  
6 Analyzing 'I like to study it':  
7 communication occurs 0 times  
8 digital occurs 0 times  
9 study occurs 1 times
```



When would you use which approach?

- Both can have a place in your workflow (e.g., bottom-up as first exploratory step)
- You have a clear theoretical expectation? Bottom-up makes little sense.
- But in any case: you need to transform your text into something “countable”.

Unsupervised Machine Learning for Text Classification



Supervised vs Unsupervised

Supervised machine learning

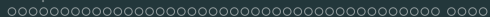
You have a dataset with both predictor and outcome (independent and dependent variables; features and labels) — a *labeled* dataset. Think of regression: You measured x_1 , x_2 , x_3 and you want to predict y , which you also measured

Unsupervised machine learning

You have no labels. (You did not measure y)

You might already know *some* techniques to figure out whether $x_1, x_2, \dots, x_i$ co-occur

- Principal Component Analysis (PCA) and Singular Value Decomposition (SVD)
- Cluster analysis
- Topic modelling (Non-negative matrix factorization and Latent Dirichlet Allocation)



Supervised vs Unsupervised

Supervised machine learning

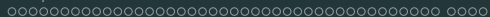
You have a dataset with both predictor and outcome (independent and dependent variables; features and labels) — a *labeled* dataset. Think of regression: You measured x_1 , x_2 , x_3 and you want to predict y , which you also measured

Unsupervised machine learning

You have no labels. (You did not measure y)

You might already know *some* techniques to figure out whether $x_1, x_2, \dots, x_i$ co-occur

- Principal Component Analysis (PCA) and Singular Value Decomposition (SVD)
- Cluster analysis
- Topic modelling (Non-negative matrix factorization and Latent Dirichlet Allocation)



Supervised vs Unsupervised

Supervised machine learning

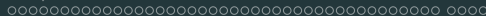
You have a dataset with both predictor and outcome (independent and dependent variables; features and labels) — a *labeled* dataset. Think of regression: You measured x_1, x_2, x_3 and you want to predict y , which you also measured

Unsupervised machine learning

You have no labels. (You did not measure y)

You might already know *some* techniques to figure out whether $x_1, x_2, \dots, x_i$ co-occur

- Principal Component Analysis (PCA) and Singular Value Decomposition (SVD)
- Cluster analysis
- Topic modelling (Non-negative matrix factorization and Latent Dirichlet Allocation)



Supervised vs Unsupervised

Supervised machine learning

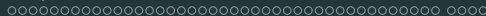
You have a dataset with both predictor and outcome (independent and dependent variables; features and labels) — a *labeled* dataset. Think of regression: You measured x_1, x_2, x_3 and you want to predict y , which you also measured

Unsupervised machine learning

You have no labels. (You did not measure y)

You might already know *some* techniques to figure out whether $x_1, x_2, \dots, x_i$ co-occur

- Principal Component Analysis (PCA) and Singular Value Decomposition (SVD)
- Cluster analysis
- Topic modelling (Non-negative matrix factorization and Latent Dirichlet Allocation)



Supervised vs Unsupervised

Supervised machine learning

You have a dataset with both predictor and outcome (independent and dependent variables; features and labels) — a *labeled* dataset. Think of regression: You measured x_1, x_2, x_3 and you want to predict y , which you also measured

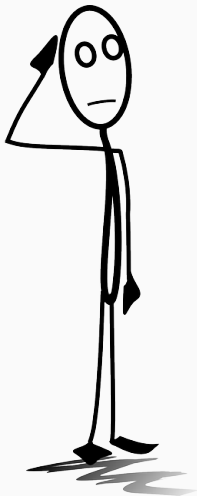
Unsupervised machine learning

You have no labels. (You did not measure y)

You might already know *some* techniques to figure out whether $x_1, x_2, \dots, x_i$ co-occur

- Principal Component Analysis (PCA) and Singular Value Decomposition (SVD)
- Cluster analysis
- Topic modelling (Non-negative matrix factorization and Latent Dirichlet Allocation)

Unsupervised ML



Recap: Can you explain the difference between supervised and unsupervised machine learning?

	Methodological approach		
	Counting and Dictionary	Supervised Machine Learning	Unsupervised Machine Learning
Typical research interests and content features	visibility analysis sentiment analysis subjectivity analysis	frames topics gender bias	frames topics
Common statistical procedures	string comparisons counting	support vector machines naive Bayes	principal component analysis cluster analysis latent dirichlet allocation semantic network analysis



do not know the topics in advance.

If you do have theoretical expectations, use classic SML (or fine-tune a Transformer, maybe with few-/zero-shot learning.) instead.

Unsupervised ML

A historical overview: PCA, k-means, LDA

1. Finding similar variables (dimension reduction)
2. Finding similar cases (clustering)

0.04.

1. Finding similar variables (dimension reduction)
2. Finding similar cases (clustering)

Are we more interested in which features “belong together” or which cases “belong together”?

1. Finding similar variables (dimension reduction)
2. Finding similar cases (clustering)

Are we more interested in which features “belong together” or which cases “belong together”?

Conceptually, we want to know *both* which features (words) belong to each other (=form a topic), *and* which cases (documents) contain the same topics.

```
1      w1,w2,w3,w4,w5,w6 ...
2  text1, 2, 0, 0, 1, 2, 3 ...
3  text2, 0, 0, 1, 2, 3, 4 ...
4  text3, 9, 0, 1, 1, 0, 0 ...
5  ...
```

1. We run a PCA/SVD to see which features (words) load on the same component; and *then* look at the component scores per document
2. We run a *k*-means cluster analysis to see which texts are similar; and *then* look at the most common words per cluster

We could then go via two routes:

1. We run a PCA/SVD to see which features (words) load on the same component; and *then* look at the component scores per document
2. We run a k -means cluster analysis to see which texts are similar; and *then* look at the most common words per cluster

- Components are ordered (first explains most variance) $\Rightarrow$ We assume that some topics are more important than others
- Components do *not* necessarily carry a meaningful interpretation $\Rightarrow$ But maybe OK in practice?
- We assume that a word belongs to one (not multiple) topics
- We assume that a document has a score for each topic

- We assume that (in the case of k -means) that topics (are roughly) similarly sized
- We assume that a *document* belongs to one (not multiple) topics
- We assume that a word *can* belong to multiple topics.

- Both have different assumptions, implications, and constraints
- Both easy to do with scikit-learn
- Both used in research (for instance, PCA by Leydesdorff and Nerghes (2017-04) or Greussing and Boomgaarden (2017); or *k*-means cluster analysis by Burscher et al. (2016))
- Typically, PCA groups features, cluster analysis groups texts – (but you can then use the component scores to describe the texts, and the cluster centroids to describe the features)
- Still occasionally used, but in general considered outdated

You find some slides with code examples in the appendix.

There is: Latent Dirichlet Allocation (LDA) (D. Blei et al., 2003).

Progress: 100%

1. Which topics can we extract from the corpus?
2. How present is each of these topics in each text in the corpus?

It also does not suffer from a few problems:

- does the goal of PCA, to find a solution in which one word loads on *one* component match real life, where a word can belong to several topics or frames?
- does the goal of cluster analysis, assigning each document to *one* cluster, match real life?

Progress: 100%

1. estimates *simultaneously* (a) which topics we find in the whole corpus, and (b) which of these topics are present in which document; while at the same time
2. allowing (a) words to be part of multiple topics, and (b) multiple topics to be present in one document; and
3. being able to make connections between words “even if they never actually occurred in a document together” (Maier et al., 2018, p. 96)

21

Progress: 100%

1. estimates *simultaneously* (a) which topics we find in the whole corpus, and (b) which of these topics are present in which document; while at the same time
2. allowing (a) words to be part of multiple topics, and (b) multiple topics to be present in one document; and
3. being able to make connections between words “even if they never actually occurred in a document together” (Maier et al., 2018, p. 96)

21

Progress: 100%

1. estimates *simultaneously* (a) which topics we find in the whole corpus, and (b) which of these topics are present in which document; while at the same time
2. allowing (a) words to be part of multiple topics, and (b) multiple topics to be present in one document; and
3. being able to make connections between words “even if they never actually occurred in a document together” (Maier et al., 2018, p. 96)

21

Progress: 100%

1. estimates *simultaneously* (a) which topics we find in the whole corpus, and (b) which of these topics are present in which document; while at the same time
2. allowing (a) words to be part of multiple topics, and (b) multiple topics to be present in one document; and
3. being able to make connections between words “even if they never actually occurred in a document together” (Maier et al., 2018, p. 96)

21

Progress: 100%

1. estimates *simultaneously* (a) which topics we find in the whole corpus, and (b) which of these topics are present in which document; while at the same time
2. allowing (a) words to be part of multiple topics, and (b) multiple topics to be present in one document; and
3. being able to make connections between words “even if they never actually occurred in a document together” (Maier et al., 2018, p. 96)

21

- There are k topics, $T_1 \dots T_k$
- Each document D_i consists of a mixture of these topics, e.g. 80% T_1 , 15% T_2 , 0% T_3 , ... 5% T_k
- On the next level, each topic consists of a specific probability distribution of words
- Thus, based on the frequencies of words in D_i , one can infer its distribution of topics
- Note that LDA (like PCA) is a Bag-of-Words (BOW) approach

0

```
1 articles=['The tax deficit is higher than expected. This said xxx ...',
           'Germany won the World Cup. After a']
2 texts=[[token for token in re.split(r"\W", art) if len(token)>0] for art
          in articles]
```

```
1 [['The', 'tax', 'deficit', 'is', 'higher', 'than', 'expected', 'This', 'said', 'xxx'], ['Germany', 'won', 'the', 'World', 'Cup', 'After', 'a']]
```

```
1 from gensim import corpora, models
2 import pandas as pd
3
4 NTOPICS = 100
5 LDAOUTPUTFILE="topicscores.tsv"
6
7 # Create a BOW representation of the texts
8 id2word = corpora.Dictionary(texts)
9 mm =[id2word.doc2bow(text) for text in texts]
10
11 # Train the LDA models.
12 mylda = models.ldamodel.LdaModel(corpus=mm, id2word=id2word, num_topics=
    NTOPICS, alpha="auto")
13
14 # Print the topics.
15 for top in mylda.print_topics(num_topics=NTOPICS, num_words=5):
16     print ("\n",top)
17
18 # the topic scores per document
19 topics = pd.DataFrame([dict(mylda.get_document_topics(doc,
    minimum_probability=0.0)) for doc in mm])
```



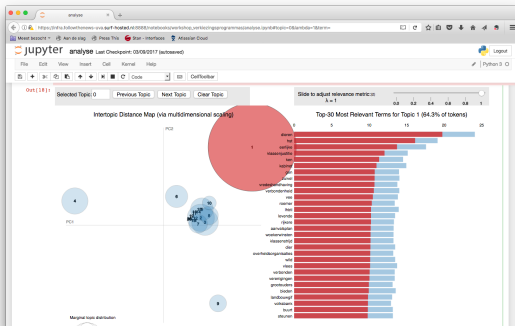
```

1  0.069*fusie + 0.058*brussel + 0.045*europese_commissie + 0.036*europese +
    0.023*overname
2  0.109*bank + 0.066*britse + 0.041*regering + 0.035*financien + 0.033*
    minister
3  0.114*nederlandse + 0.106*nederland + 0.070*bedrijven + 0.042*rusland +
    0.038*ruisische
4  0.093*nederlandsespoorwegen + 0.074*den + 0.036*jaar + 0.029*onderzoek +
    0.027*raad
5  0.099*banen + 0.045*jaar + 0.045*productie + 0.036*ton + 0.029*aantal
6  0.041*grote + 0.038*bedrijven + 0.027*ondernemers + 0.023*goed + 0.015*
    jaar
7  0.108*werknemers + 0.037*jongeren + 0.035*werkgevers + 0.029*jaar +
    0.025*werk
8  0.171*bank + 0.122* + 0.041*klanten + 0.035*verzekeraar + 0.028*euro
9  0.162*banken + 0.055*bank + 0.039*centrale + 0.027*leningen + 0.024*
    financiële
10 0.052*post + 0.042*media + 0.038*nieuwe + 0.034*netwerk + 0.025*
    personeel
11 ...

```

Data Editor (Browse) - topicscores.data													
topic4[2]		.019											
source2	firstwords	polarity	subjectivity	pubdate_day	pubdate_mo-h	pubdate_year	pubdate_da-k	topic1	topic2	topic3	topic4	topic5	
1	nrc handelsblad	palingsound schinke	-.0086207	.6069971	31	12	2011	zaterdag	.018	.019	3.587	.019	.019
2	nrc handelsblad	groep investeerders	-.1041667	.3129194	31	12	2011	zaterdag	.018	.019	.019	.019	.019
3	nrc handelsblad	abnamro debacles ij	.0082292	.4895443	31	12	2011	zaterdag	.018	27.71	.019	.019	.019
4	nrc handelsblad	abnamro financi' le	-.0179617	.5706419	31	12	2011	zaterdag	.018	15.1	.019	2.646	.019
5	nrc handelsblad	crisis verhouding k	.0758049	.5448864	31	12	2011	zaterdag	.018	.019	9.008	.019	.019
6	nrc handelsblad	snel vakantie vrije	-.016315	.5118008	31	12	2011	zaterdag	.018	.019	.019	.019	.019
7	nrc handelsblad	herinnering doos le	.18875	.6200333	31	12	2011	zaterdag	.018	.019	.019	.019	.019
8	nrc handelsblad	hackers publiceren	.1454545	.4545455	31	12	2011	zaterdag	.018	.019	.019	.019	.019
9	nrc handelsblad	waterballet nontevi	-.2333333	.4333333	31	12	2011	zaterdag	.018	.019	.019	.019	.019
10	nrc handelsblad	bouw dupe ambities	.0925417	.5939167	5	11	2010	vrijdag	.018	.019	.078	2.442	.019
11	nrc handelsblad	eindelijk wint nuh	.1755093	.48125	5	11	2010	vrijdag	.018	.019	8.302	.019	.019
12	nrc handelsblad	oud nieuws tv bbct	.02	.4322222	5	11	2010	vrijdag	.018	10.053	.019	.019	.019
13	nrc handelsblad	tag hyves krantenb	.0425203	.5420412	5	11	2010	vrijdag	.018	.019	.019	.019	.019
14	nrc handelsblad	getuigenis rechter	.0858929	.5770833	5	11	2010	vrijdag	.018	.019	.019	11.621	.019
15	nrc handelsblad	akzonobel philips g	.0220455	.4381818	5	11	2010	vrijdag	.018	.019	.019	.019	.019
16	nrc handelsblad	mondiaal kritiek be	-.038172	.3894624	5	11	2010	vrijdag	.018	19.957	.019	.019	.019
17	nrc handelsblad	export diamant fiat	.0628571	.4438895	5	11	2010	vrijdag	.018	4.745	.019	.019	.019
18	nrc handelsblad	canada bod potash r	.0252924	.4795322	5	11	2010	vrijdag	.018	26.741	.019	.019	.019
19	nrc handelsblad	zwakke bouwsector c	.0171	.4736333	14	3	2009	NA	.018	.019	.019	.019	4.806
20	nrc handelsblad	pensioenconflict wa	.028114	.4636842	14	3	2009	NA	.018	.019	.019	.019	.019
21	nrc handelsblad	rechter allin loon	.1318182	.3939394	14	3	2009	NA	.018	.019	.019	.019	.019
22	nrc handelsblad	bad bank remedie da	.0891026	.550641	14	3	2009	NA	.018	10.235	.019	.019	.019
23	nrc handelsblad	bescheiden salaris	-.075	.56	14	3	2009	NA	.018	.019	.019	.019	.019
24	nrc handelsblad	generalmotors autos	.0138889	.4388889	14	3	2009	NA	.018	.019	.019	.019	.019
25	nrc handelsblad	rusland rozen tuinb	.0314141	.5643051	14	3	2009	NA	.018	.019	24.595	.019	.019
26	nrc handelsblad	cynisae oplossing k	.0100833	.6511667	14	3	2009	NA	.018	.019	.019	.019	.019
27	nrc handelsblad	the good bed ugly l	.0265504	.5298449	13	3	2009	NA	.018	.019	.019	.019	.019
28	nrc handelsblad	kerk stroom nietswe	-.0087719	.6149123	13	3	2009	NA	.018	.019	.019	.019	.019
29	nrc handelsblad	kerk stroom goud ac	0	0	13	3	2009	NA	.018	.019	.019	.019	.019
30	nrc handelsblad	supersnelle koekn	0	0	13	3	2009	NA	.018	.019	.019	.019	.019
31	nrc handelsblad	dalailama chinese e	0	0	13	3	2009	NA	.018	.019	.019	.019	.019
32	nrc handelsblad	bezuinigen hulpgeld	.0894192	.4560606	13	3	2009	NA	.018	.019	.019	.019	.019
33	nrc handelsblad	vaders arbeidsethos	.0160985	.5575758	13	3	2009	NA	.018	.019	.019	.019	.019
34	nrc handelsblad	varkens lux winnaar	.040073	.6218254	4	10	2008	NA	.018	.019	.019	.019	.019
35	nrc handelsblad	liberale kinderopva	.1179095	.5297055	4	10	2008	NA	.018	.019	.019	.019	1.83
36	nrc handelsblad	banken verzinsels k	.068521	.6308389	4	10	2008	NA	8.232	.019	.019	.019	.019
37	nrc handelsblad	rabobanktopman bert	0	0	4	10	2008	NA	.018	.019	.019	.019	.019
38	nrc handelsblad	kinderopvang bril v	0	0	4	10	2008	NA	.018	.019	.019	.019	.019
39	nrc handelsblad	tassen gevoel verli	0	0	4	10	2008	NA	.018	.019	.019	.019	.019
40	nrc handelsblad	abnamro winklend p	.0876761	.62277	4	10	2008	NA	.018	.019	6.904	.019	5.511
41	nrc handelsblad	abnamro belgi' mole	.0439506	.4976852	4	10	2008	NA	.018	.019	.019	.019	.019
42	nrc handelsblad	abnamro handen deut	.1838401	.5264302	4	10	2008	NA	.018	.019	1.854	.019	.019
43	nrc handelsblad	abnamro fortis bank	.0842391	.494058	4	10	2008	NA	4.939	.019	14.39	.019	.019
44	nrc handelsblad	abnamro fortis spra	.0540715	.6290807	4	10	2008	NA	.018	.019	.019	.019	.019
45	nrc handelsblad	abnamro fortis jaar	.0297297	.4960135	4	10	2008	NA	.018	11.041	.019	.019	.019
46	nrc handelsblad	abnamro nederland s	.1006944	.6830555	4	10	2008	NA	.018	.019	.019	.019	.019
47	nrc handelsblad	abnamro belgi' mole	.0405952	.5804464	4	10	2008	NA	.018	.019	.019	.019	.019
48	nrc handelsblad	arbeidsmarkt vs sle	.0166667	.4	4	10	2008	NA	7.103	.019	.019	.019	12.682

```
1 import pyLDAvis
2 import pyLDAvis.gensim_models as gensimvis
3 # first estimate gensim model, then:
4 vis_data = gensimvis.prepare(mylda,mm,id2word)
5 pyLDAvis.display(vis_data)
```



19

Short note about the λ setting:

It influences the ordering of the words in `pyldavis`.

“For $\lambda = 1$, the ordering of the top words is equal to the ordering of the standard conditional word probabilities. For λ close to zero, the most specific words of the topic will lead the list of top words. In their case study, Sievert and Shirley (2014, p. 67) found the best interpretability of topics using a λ -value close to .6, which we adopted for our own case” (Maier et al., 2018, p. 107)

Evaluation metrics (closer to zero is better)

perplexity

A goodness-of-fit measure, answering the question: If we do a train-test split, how well does the trained model fit the test data?

A **L**

- Basically, similar to the idea behind our grid search from two weeks ago: estimate multiple models, store the metrics for each model, and then compare them (numerically, or by plotting)
- Idea: We select some candidate models, and then look whether they can be interpreted.
- But what can we tune?

Choosing k : How many topics do we want?

- Typical values: $10 < k < 200$
- Too low: losing nuance, so broad it becomes meaningless
- Too high: picks up tiny peculiarities instead of finding general patterns
- There is no inherent ordering of topics (unlike PCA!)
- We can throw away or merge topics later, so if out of $k = 50$ topics 5 are not interpretable and a couple of others overlap, it still may be a good model

Choosing α : how sparse should the document-topic distribution θ be?

- The higher α , the more topics per document
- Default: $1/k$
- But: We can explicitly change it, or – really cool – even learn α from the data (`alpha = "auto"`)

Takeaway: It takes longer, but you probably want to learn alpha from the data, using multiple passes:

```
1 mylda LdaModel(corpus=tfidfcorpus[ldacorpus], id2word=id2word,  
    num_topics=50, alpha='auto', passes=10)
```

Choosing α : how sparse should the document-topic distribution θ be?

- The higher α , the more topics per document
- Default: $1/k$
- But: We can explicitly change it, or – really cool – even learn α from the data (`alpha = "auto"`)

Takeaway: It takes longer, but you probably want to learn α from the data, using multiple passes:

```
1 mylda LdaModel(corpus=tfidfcorpus[ldacorpus], id2word=id2word,  
    num_topics=50, alpha='auto', passes=10)
```

Choosing η : how sparse should the topic-word distribution λ be?

- Can be used to boost specific words
- Can also be learned from the data

Takeaway: Even though you can do `eta="auto"`, this usually does not help you much.

Choosing η : how sparse should the topic-word distribution λ be?

- Can be used to boost specific words
- Can also be learned from the data

Takeaway: Even though you can do `eta="auto"`, this usually does not help you much.

Using topic models

You got your model – what now?

1. Assign topic scores to documents
2. Label topics
3. Merge topics, throw away boilerplate topics and similar (manually, or aided by cluster analysis)
4. Compare topics between, e.g., outlets
5. or do some time-series analysis.

Example: Tsur et al., 2015

Unsupervised ML

Should one still use LDA?

The popularity of LDA

In the last decade, LDA has become *extremely* popular in the social sciences due to

- easy-to-use R and Python packages
- its promise to not require (a) manual (qual or quant) analysis; (b) annotations for SML; (c) creation of dictionaries etc.
- a bit of a “cool new technique” image

Progress: 100%

There are some extensions on classical LDA, in particular:

- Author-topic models
- Structural topic models (STM) (Roberts et al., 2014)
- Dynamic topic models (D. M. Blei & Lafferty, 2006)

These allow covariates (e.g., add info on who wrote a text) to improve the model, or allow to account for the changing use of words and topics over time.

Also, there are techniques for validation available (e.g., topic intrusion and/or word intrusion tasks).

Unsupervised Machine Learning for Text Classification

○○

Unsupervised ML

State-of-the-art approaches to topic modelling

Let's bring in embeddings and Transformers!

Progress: 100%

- top2vec (Angelov, 2020), which embeds *topic vectors* in the same space as document vectors and word vectors
- Contextualized Topic models (Bianchi et al., 2021-08; Bianchi et al., 2021-04), with a lot of code examples at <https://contextualized-topic-models.readthedocs.io/en/latest/introduction.html>

“In this paper, we introduce BERTopic, a topic model that leverages clustering techniques and a class-based variation of TF-IDF to generate coherent topic representations. More specifically, we first create document embeddings using a pretrained language model to obtain document-level information. Second, we first reduce the dimensionality of document embeddings before creating semantically similar clusters of documents that each represent a distinct topic. Third, to overcome the centroid-based perspective, we develop a classbased version of TF-IDF to extract the topic representation from each topic. These three independent steps allow for a flexible topic model that can be used in a variety of use-cases, such as dynamic topic modeling.”

(for details, read the paper)

	20 NewsGroups		BBC News		Trump	
	TC	TD	TC	TD	TC	TD
LDA	.058	.749	.014	.577	-.011	.502
NMF	.089	.663	.012	.549	.009	.379
T2V-MPNET	.068	.718	-.027	.540	-.213	.698
T2V-Doc2Vec	.192	.823	.171	.792	-.169	.658
CTM	.096	.886	.094	.819	.009	.855
BERTopic-MPNET	.166	.851	.167	.794	.066	.663

Table 1: Ranging from 10 to 50 topics with steps of 10, topic coherence (TC) and topic diversity (TD) were calculated at each step for each topic model. All results were averaged across 3 runs for each step. Thus, each score is the average of 15 separate runs.

(And no need to set k ! And there is a dedicated “outlier topic” called -1 !)

- By definition, much more “black-box”-y than BOW approaches
- Risk of biases introduced by LLM
- Much more resource-hungry (you probably want to do this with a GPU (e.g., on CoLab))

To conclude: PCA, k -means, LDA are interesting starting points – but if I were to start an unsupervised topic analysis model now, I'd go for BERTopic.

From Bag-of-Words to Embeddings

- Goal in topic modelling:
 - Group documents that **talk about similar things**.
- Bag-of-Words relies on exact word overlap.
- We want a representation where:
 - Similar texts are close together.
 - Dissimilar texts are far apart.
- Enter: **Embeddings**.

Embeddings

What Are Embeddings?

- Dense numerical vectors representing meaning.
- Each document becomes a vector with, e.g., 384 dimensions.
- Core idea:

Semantic Proximity

Documents with similar meaning have similar vectors.

Polysemy and Context

- **Polysemy:** one word, multiple meanings.
- Example: “bank” (finance) vs “bank” (river).
- Bag-of-Words cannot distinguish these.
- Embeddings consider context, so the vectors differ.

Dense vs Sparse Representations

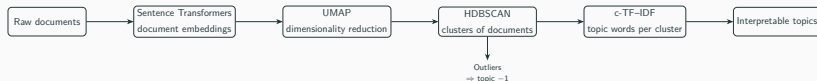
- Bag-of-Words vectors:
 - very high dimensional,
 - mostly zeros (**sparse**).
- Embeddings:
 - low dimensional,
 - mostly non-zero (**dense**).
- Dense vectors make similarity and clustering easier.

From Embeddings to BERTopic

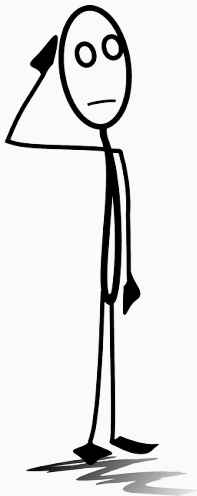
- Capture context, synonyms, and meaning.
- Similar texts are numerically close.
- Better suited for clustering than Bag-of-Words.
- Provides a strong basis for modern topic models.

BERTopic: High-Level Idea

- Classic models (e.g. LDA) use Bag-of-Words.
- **BERTopic** uses embeddings instead.
- Then:
 1. reduce dimensions,
 2. cluster documents,
 3. extract topic words.
- All steps run automatically in BERTopic.



In practice, running `BERTopic()` executes all steps automatically.



Any questions?

Next steps

Note: I would advise to run the BERTopic model on Google Colab so you have access to a GPU. If there is time, we can discuss this together in class.

References



Angelov, D. (2020). Top2vec: Distributed representations of topics. *arXiv preprint arXiv:2008.09470*.



Bianchi, F., Terragni, S., & Hovy, D. (2021-08). Pre-training is a hot topic: Contextualized document embeddings improve topic coherence. *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 2: Short Papers)*, 759–766. <https://doi.org/10.18653/v1/2021.acl-short.96>



Bianchi, F., Terragni, S., Hovy, D., Nozza, D., & Fersini, E. (2021-04). Cross-lingual contextualized topic models with zero-shot learning. *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, 1676–1683. <https://www.aclweb.org/anthology/2021.eacl-main.143>



Blei, D. M., & Lafferty, J. D. (2006). Dynamic topic models. *Proceedings of the 23rd international conference on Machine learning*, 113–120.



Blei, D., Ng, A., & Jordan, M. (2003). Latent dirichlet allocation. *Journal of machine Learning research*, 3, 993–1022.



Boumans, J. W., & Trilling, D. (2016). Taking stock of the toolkit: An overview of relevant automated content analysis approaches and techniques for digital journalism scholars. *Digital Journalism*, 4(1), 8–23. <https://doi.org/10.1080/21670811.2015.1096598>



Burscher, B., Vliegthart, R., & de Vreese, C. H. (2016). Frames beyond words: Applying cluster and sentiment analysis to news coverage of the nuclear power issue. *Social Science Computer Review*, 34(5), 530–545. <https://doi.org/10.1177/0894439315596385>



Greussing, E., & Boomgaarden, H. G. (2017). Shifting the refugee narrative? An automated frame analysis of Europe's 2015 refugee crisis. *Journal of Ethnic and Migration Studies*, 43(11), 1749–1774. <https://doi.org/10.1080/1369183X.2017.1282813>



Grootendorst, M. (2022). Bertopic: Neural topic modeling with a class-based tf-idf procedure. *arXiv preprint arXiv:2203.05794*.



Leydesdorff, L., & Nerghe, A. (2017-04). Co-word maps and topic modeling: A comparison using small and medium-sized corpora ($N < 1,000$) [arXiv: 0803.1716 ISBN: 9783848215430]. *Journal of the Association for Information Science and Technology*, 68(4), 1024–1035. <https://doi.org/10.1002/asi.23740>



Maier, D., Waldherr, A., Miltner, P., Wiedemann, G., Niekler, A., Keinert, A., Pfetsch, B., Heyer, G., Reber, U., Häussler, T., Schmid-Petri, H., & Adam, S. (2018). Applying LDA topic modeling in communication research: Toward a valid and reliable methodology. *Communication Methods and Measures*, 12(2-3), 93–118. <https://doi.org/10.1080/19312458.2018.1430754>



Řehůřek, R., & Sojka, P. (2010). Software framework for topic modelling with large corpora [http://is.muni.cz/publication/884893/en]. *Proceedings of the LREC 2010 Workshop on New Challenges for NLP Frameworks*, 45–50.



Roberts, M. E., Stewart, B. M., Tingley, D., Lucas, C., Leder-Luis, J., Gadarian, S. K., Albertson, B., & Rand, D. G. (2014). Structural topic models for open-ended survey responses. *American journal of political science*, 58(4), 1064–1082.

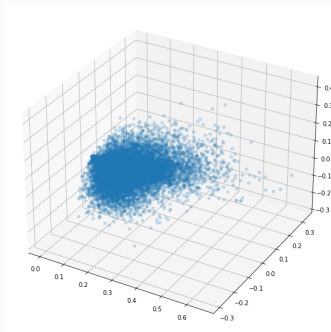


Tsur, O., Calacci, D., & Lazer, D. (2015). A Frame of Mind: Using Statistical Models for Detection of Framing and Agenda Setting Campaigns. *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing*, 1629–1638.

Appendix

```
1 from sklearn import datasets
2 from sklearn.decomposition import TruncatedSVD
3 from sklearn.feature_extraction.text import TfidfVectorizer
4 from sklearn.pipeline import make_pipeline
5
6 texts = datasets.fetch_20newsgroups(data_home='rec.autos',
   ↪ remove=('headers', 'footers', 'quotes'), subset='train')['data']
7
8 myvec = TfidfVectorizer(max_df=.5, min_df=5,
   ↪ token_pattern='(?u)\\b[a-zA-Z][a-zA-Z]+\\b')
9 mysvd = TruncatedSVD(n_components=3)
10 mypipe = make_pipeline(myvec, mysvd)
11 r = mypipe.fit_transform(texts)
```

```
1 import matplotlib.pyplot as plt
2
3 fig = plt.figure(figsize=(10,10))
4 ax = fig.add_subplot(projection='3d')
5 ax.scatter([e[0] for e in r], [e[1] for e in r], [e[2] for e in r], alpha
            =.2)
```



Using the scores

```

1 import pandas as pd
2 textscores= pd.DataFrame(r)
3 featurescores = pd.DataFrame(mysvd.components_.T, index = myvec.
    get_feature_names())

```

featurescores				textscores			
	0	1	2		0	1	2
aa	0.001019	0.001665	-0.001053	0	0.252318	-0.048142	-0.100314
aaa	0.001799	0.003467	-0.002502	1	0.142955	-0.074158	0.006181
aaron	0.000583	0.001025	-0.000948	2	0.320513	-0.104095	-0.088318
ab	0.000994	0.001040	-0.002188	3	0.179523	-0.086744	0.088419
abandon	0.000719	0.000626	0.000212	4	0.156641	-0.003298	0.030814
...	...	...	...	...	...	...	...
zur	0.000075	0.000344	-0.000342	11309	0.205703	-0.002241	0.030386
zv	0.000066	-0.000134	-0.000123	11310	0.183100	-0.096423	-0.070144
zx	0.001135	-0.000953	0.000787	11311	0.129721	-0.011929	-0.031808
zy	0.000021	-0.000080	-0.000075	11312	0.159569	-0.016293	0.039790
zz	0.000110	-0.000156	-0.000034	11313	0.086385	-0.041932	-0.034792

16150 rows × 3 columns

11314 rows × 3 columns

Figure 1

- We used SVD to figure out relationships between features
- We could now look at the most important features per component (“topic”, “frame”?) by sorting `featurescores`
- We could see which texts are most representative for each “topic” or “frame” by sorting `textscores`

Grouping features vs grouping cases

We have a corpus of a many texts.

- We used SVD to figure out relationships between features
- We could now look at the most important features per component (“topic”, “frame”?) by sorting `featurescores`
- We could see which texts are most representative for each “topic” or “frame” by sorting `textscores`

⇒ Alternative: Choose the opposite approach and first find out which cases are most similar, *then* describe what features characterize each group of cases

```
1 from sklearn.cluster import KMeans
2
3 k = 5
4
5 mykm = KMeans(n_clusters=k, init='k-means++', max_iter=100, n_init=1)
6 myvec = TfidfVectorizer(max_df=.5, min_df=5,
7 ↪ token_pattern='(?u)\\b[a-zA-Z][a-zA-Z]+\\b')
8 mypipe = make_pipeline(myvec, mykm)
9
10 predictions = mypipe.fit_transform(texts)
11 # potentially: textscores = pd.DataFrame(predictions)
```

Of course, you need to determine the appropriate k ... (see earlier lecture)

Let's get the terms closest to the centroids

```
1 order_centroids = mykm.cluster_centers_.argsort()[:, :-1]
2 terms = myvec.get_feature_names()
3
4 print("Top terms per cluster:")
5
6 for i in range(k):
7     print("Cluster {}: ".format(i), end='')
8     for ind in order_centroids[i, :10]:
9         print("{} ".format(terms[ind]), end='')
10    print()
```

returns something like:

```
1 Top terms per cluster:
2 Cluster 0: windows file dos window with on you this have files
3 Cluster 1: you on this was with are have be not they
4 Cluster 2: thanks any me have anyone or please if on this
5 Cluster 3: he was his him not as this but on god
6 Cluster 4: you are be not they as this have if on
```

(of course, we could do sth similar with pandas as well)

Let's get the terms closest to the centroids

```
1 order_centroids = mykm.cluster_centers_.argsort()[:, :-1]
2 terms = myvec.get_feature_names()
3
4 print("Top terms per cluster:")
5
6 for i in range(k):
7     print("Cluster {}: ".format(i), end='')
8     for ind in order_centroids[i, :10]:
9         print("{} ".format(terms[ind]), end='')
10    print()
```

returns something like:

```
1 Top terms per cluster:
2 Cluster 0: windows file dos window with on you this have files
3 Cluster 1: you on this was with are have be not they
4 Cluster 2: thanks any me have anyone or please if on this
5 Cluster 3: he was his him not as this but on god
6 Cluster 4: you are be not they as this have if on
```

(of course, we could do sth similar with pandas as well)